

Document Number:	P0912R5
Date:	2019-02-22
Audience:	WG21
Revises:	P0912R4
Reply to:	gorn@microsoft.com

Merge Coroutines TS into C++20 working draft

Abstract

This paper proposes merging Working Draft of Coroutines TS [N4775] into the C++20 working draft [N4800].

Revision history

r0: initial revision

r1:

- Markdown rendered as HTML.
- Replaced ligature ff with two letters ff.
- Expanded on compiler availability.
- Reworded the closing paragraph.
- Make insertions **green**.
- Updated working draft numbers

r2:

- ... keywords s/were/are added ...
- P0664R3 => P0664R4
- Added list of edits requested by LWG on 2018-06-08.

r3:

- document numbers and edition instructions updated

r4:

- document numbers and edition instructions updated

r5:

- incorporated the resolutions to Coroutines Issues #25 #27 (<https://wg21.link/P1356R0>)

- incorporated the resolution to Coroutines Issues #31 #35 (<https://wg21.link/P0664R7>)

Introduction

- Coroutines of this kind were available in a shipping compiler since 2014.
- We have been receiving and acting upon developer feedback for 5 years.
- The software built using coroutines has been deployed on more than **400 million** devices in the hands of delighted consumers.
- The software built using coroutines on Linux and Windows is powering the foundational services of Windows Azure cloud services.
- Coroutines have been used by thousands of software developers in various companies.
- We have two publicly available implementations of coroutines TS in MSVC since version 2015 SP2 (2016) and clang version 5 (2017).
- We have an independent coroutine implementation in EDG frontend (2015).
- GCC implementation is in progress.
- There are several open source coroutine abstraction libraries, including an extensive open source coroutine library [cppcoro](#) that utilizes most of the coroutine features with tests that can be used to smoke test emerging coroutine implementation for completeness and correctness.
- Other libraries provide coroutines TS bindings for its types, for example, Facebook's Folly, Just::Thread Pro and others.

Coroutines address the dire need by dramatically simplifying development of asynchronous code. Coroutines have been available and in use for 5 years. We have shipping implementations from two major compiler vendors. It is time to merge Coroutines TS to the working paper.

Wording

- Apply coroutine wording from N4775 to the working draft with the following changes:
 - replace `experimental::` with nothing
 - replace `<experimental/coroutine>` with `<coroutine>`
 - in synopsis in `[coroutine.trivial.awaitables]` and `[support.coroutine]`
 - remove "namespace experimental {" and "} // namespace experimental"
 - remove "inline namespace coroutines_v1 {" and "} // namespace coroutines_v1"
 - exclude changes to `[stmt.iter]` and `[stmt.ranged]` (issue #35 incorporated)
 - LaTeX specific instructions: before applying the changes, in the coroutine TS LaTeX source, replace all occurrences of `\cxxref{stable-name}` with `\ref{stable-name}`.
- Add underlined text to rationale in `[diff.cpp17.lex]`:

Rationale: Required for new features. The `requires` keyword is added to introduce constraints through a *requires-clause* or a *requires-expression*. The `concept` keyword is added to enable the definition of concepts (12.6.8). The `co await`, `co yield`, and `co return` keywords are added to enable the definition of coroutines ([\ref{dcl.fct.def.coroutine}](#)). Effect

on original feature: Valid ISO C++ 2017 code using concept, co await, co yield, co return, or requires as an identifier is not valid in this International Standard.

- Add underlined text to "Effect on original feature" in [diff.cpp17.library]:

Effect on original feature: The following C++ headers are new: <compare>, <coroutine>, and <syncstream>. Valid C++ 2017 code that #includes headers with these names may be invalid in this International Standard.

- Apply the following modifications that implement the resolutions of issues 25, 27, and 31
- modify [expr.await]/2 as follows:

An *await-expression* shall appear only in a potentially-evaluated expression within the *compound-statement* of a *function-body* outside of a *handler* (Clause 18). In a declaration-statement or in the simple-declaration (if any) of a for-init-statement, an ~~await-expression~~ *await-expression* shall appear only in an initializer of that declaration-statement or simple-declaration. An *await-expression* shall not appear in the initializer of a block-scope variable with static or thread storage duration.

- in section [dcl.fct.def.coroutine] add two paragraphs after paragraph 11:

12. If the evaluation of the expression *p*.unhandled_exception() exits via an exception, the coroutine is considered suspended at the final suspend point.

13. The expression co await *p*.final_suspend() shall not be potentially-throwing ([except.spec]).

- add the underlined text at the beginning of [coroutine.handle.resumption]:

1. Resuming a coroutine via resume_operator(), or destroy on an execution agent other than the one it was suspended on has implementation-defined behavior unless each is either an instance of std::thread or the thread that executes main.

[Note: A coroutine that is resumed on a different execution agent should avoid relying on consistent thread identity throughout, such as holding a mutex object across a suspend point. — end note.]

[Note: A concurrent resumption of the coroutine may result in a data race. — end note.]

- remove all occurrences of this note from [coroutine.handle.resumption]:

~~[Note: A concurrent resumption of the coroutine via resume_operator(), or destroy may result in a data race. — end note.]~~